

Ex6_Regularization2

November 24, 2025

1 Exercise Sheet 6 - Regularization 2

In this exercise we will take a look at simple but effective methods used to combat overfitting when learning neural networks. We will use a very small subset of the FashionMNIST dataset to artificially induce overfitting, train and evaluate our model. Finally we will look at how to incorporate early stopping and how adding noise to our data makes our model more robust.

```
[2]: from typing import Callable
      from functools import partial
      from IPython.display import clear_output
      import matplotlib.pyplot as plt

      import torch
      from torch import nn
      from torch.optim import Optimizer, SGD

      from torchvision.transforms import v2
      from torch.utils.data import DataLoader

      # from solution import *
      import utils

      DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
      DEVICE
```

```
[2]: 'cpu'
```

2 Training configuration

Throughout this notebook we will use the following training configuration:

```
[4]: TRAIN_SET_SIZE = 200
      VAL_SET_SIZE = 1000
      EPOCHS = 500
      HIDDEN_DIMS = [64, 32]
      LR = 0.05
      BATCH_SIZE = 32
```

3 Dataset (FashionMNIST)

As mentioned before, we will use the FashionMNIST dataset. This dataset behaves exactly the same as the standard MNIST dataset (grayscale images with height and width of 28 pixels, 10 classes (0-9), train set with 60k samples and test set with 10k samples) with the only difference being the depicted images. While MNIST shows handwritten digits, FashionMNIST shows 10 different types of clothing. Example images are shown after execution of the following cell.

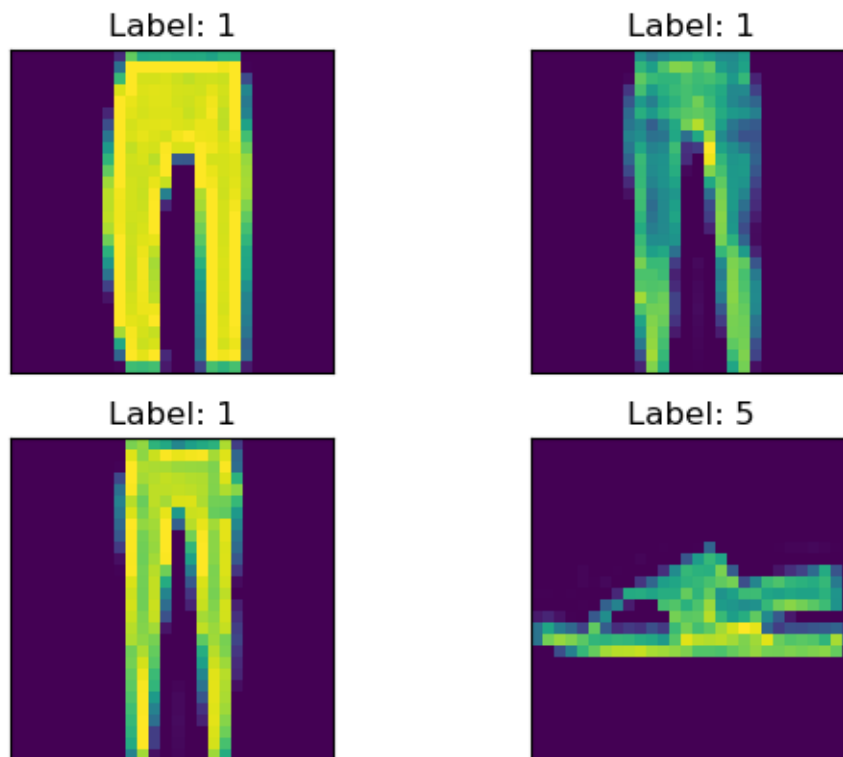
```
[6]: train_set, val_set, test_set = utils.get_fashion_mnist_subset(TRAIN_SET_SIZE, VAL_SET_SIZE, transforms=v2.ToTensor())

train_loader = DataLoader(train_set, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_set, batch_size=BATCH_SIZE)
test_loader = DataLoader(test_set, batch_size=BATCH_SIZE)

utils.visualize_first_4(train_loader)
```

```
/opt/anaconda3/lib/python3.12/site-packages/torchvision/transforms/v2/_deprecated.py:42: UserWarning: The transform `ToTensor()` is deprecated and will be removed in a future release. Instead, please use `v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32, scale=True)])`. Output is equivalent up to float precision.
  warnings.warn(
```

```
Shape of images is torch.Size([32, 1, 28, 28])
```



4 Create model

First, we need a function to create a model that is able to classify FashionMNIST data. The model takes in inputs of the shape (batch_size x 1 x 28 x 28) and outputs a 10-dimensional vector. It should first flatten the input images, then apply a given number of linear layers to it, and finally map to a 10-dimensional vector which will be used to predict which type of clothing it is.

4.1 Task 1a) (20 P)

Complete the missing code in the following function. The function takes a list of hidden dimensions which correspond to the dimensionality of the linear layers. The input dimension of 28 x 28 and the output dimension of 10 should be hardcoded into the model. There should be as many layers as hidden dimensions, plus a final output layer. As an activation function we will use the ReLU activation.

```
[9]: def create_model(hidden_dims: list[int]):  
    """Create a model that works for classifying the FashioMNIST dataset."""  
    # -----  
    # TODO: Replace by your code  
    # -----  
    model = SimpleMLP(hidden_dims)  
    # -----  
  
    model.to(DEVICE)  
    return model  
  
class SimpleMLP(nn.Module):  
  
    def __init__(self, dims: list[int]):  
        super().__init__()  
  
        # -----  
        # TODO: Replace by your code  
        # -----  
        input_layer = nn.Linear(28 * 28, dims[0], bias=True)  
        output_layer = nn.Linear(dims[-1], 10, bias=True)  
        self.linears = nn.ModuleList([input_layer] + [nn.Linear(dims[i],  
↪dims[i+1], bias=True) for i in range(len(dims)-1)] + [output_layer])  
        self.activation = nn.ReLU()  
        self.flatten = nn.Flatten()  
  
        # -----  
  
    def forward(self, input: torch.Tensor):
```

```

# -----
# TODO: Replace by your code
# -----
out = self.flatten(input)
for linear in self.linears[:-1]:
    out = self.activation(linear(out))

return self.linears[-1](out)

# -----

```

5 Train, evaluate and save models

5.1 Task 2a) (20 P)

Write a function that trains a model for one epoch / one iteration through the train dataset. Make sure to zero the gradients before each step and to apply the optimizers functions to train the model. This is a repetition from last exercise, so check there in case you are unsure.

```

[12]: from typing import Callable

def train_one_epoch(model, dataloader, optimizer):
    # -----
    # TODO: Replace by your code
    # -----
    loss_fn = nn.CrossEntropyLoss()
    losses = []
    for batch, labels in dataloader:
        optimizer.zero_grad()
        out = model(batch)
        loss = loss_fn(out, labels)
        loss.backward()
        optimizer.step()
        losses.append(loss.item())
    # -----

    return sum(losses)/len(losses)

```

5.2 Task 2b) (20 P)

Write a function that iterates through a dataloader, and outputs the average loss and accuracy. Make sure that no gradient computation is triggered, e.g. use `torch.no_grad()`. Furthermore, make sure that your model is in evaluation mode and to switch back afterwards. This will be important for the last task where we are adding Dropout layers to our neural network. For more information see [this StackOverflow question](#).

```
[14]: def validate(model: nn.Module, dataloader: DataLoader) -> tuple[float, float]:
    # -----
    # TODO: Replace by your code
    # -----
    model.eval()
    loss_fn = nn.CrossEntropyLoss()
    losses = []
    n_correctly_classified = 0
    with torch.no_grad():
        for batch, labels in dataloader:

            outputs = model(batch)
            predictions = outputs.argmax(dim=1)

            # Accuracy
            n_correctly_classified += (predictions == labels).sum().item()

            # Loss
            loss = loss_fn(outputs, labels)
            losses.append(loss.item())
            # -----

    avg_accuracy = n_correctly_classified / len(dataloader.dataset)
    avg_loss = sum(losses)/len(losses)
    model.train()
    # -----
    return avg_loss, avg_accuracy
```

Here we define our complete training function. It simply iterates for `n_epochs` epochs through the training dataset, evaluates after each epoch on the validation dataset, and finally returns an array with the train and validation losses for each epoch. An important feature of this training function is that it can take a list of functions as an argument (the `callbacks` argument) where each gets - the model, - the current epoch, - the array of train losses up until this point, - the array of validation losses until this point and - the last validation accuracy.

The following tasks will partly consist of writing functions that we will pass into the training function.

```
[16]: def train(
    model: nn.Module,
    train_loader: DataLoader,
    val_loader: DataLoader,
    optimizer: Optimizer,
    n_epochs: int,
    callbacks: list[Callable[[nn.Module, int, list[float], list[float],
↪float], None]]
) -> tuple[list[float], list[float]]:
    train_losses = []
```

```

val_losses = []
for epoch in range(n_epochs):
    train_loss = train_one_epoch(model, train_loader, optimizer)
    val_loss, val_acc = validate(model, val_loader)

    train_losses.append(train_loss)
    val_losses.append(val_loss)

    for c in callbacks:
        c(model, epoch, train_losses, val_losses, val_acc)

return train_losses, val_losses

```

5.3 Task 2c) (10 P)

In this task you should write a function that we can pass into the `train` function above (as the callback argument) that plots the validation and training loss until the current epoch.

```

[18]: import matplotlib.pyplot as plt

def plot_train_and_val_loss(
    epoch: int,
    train_losses: list[float],
    val_losses: list[float],
    val_acc: float,
):
    clear_output(wait=True)
    plt.clf()
    plt.plot(train_losses, label="Train Loss")
    plt.plot(val_losses, label="Validation Loss")
    avg_train_loss = sum(train_losses)/len(train_losses)
    avg_val_loss = sum(val_losses)/len(val_losses)
    plt.title(f"Epoch {epoch} | train loss = {avg_train_loss:.2f} | val loss = {avg_val_loss:.2f} | val accuracy = {val_acc:.2f}")
    plt.hlines(min(val_losses), 1, epoch, linestyle='dashed', colors='r')

    plt.legend()

    # -----
    # TODO: Replace by your code
    # -----

    # -----

def plot_train_and_val_loss_wrapper(
    model: nn.Module,

```

```

    epoch: int,
    train_losses: list[float],
    val_losses: list[float],
    val_acc: float,
):
    plot_train_and_val_loss(epoch, train_losses, val_losses, val_acc)

```

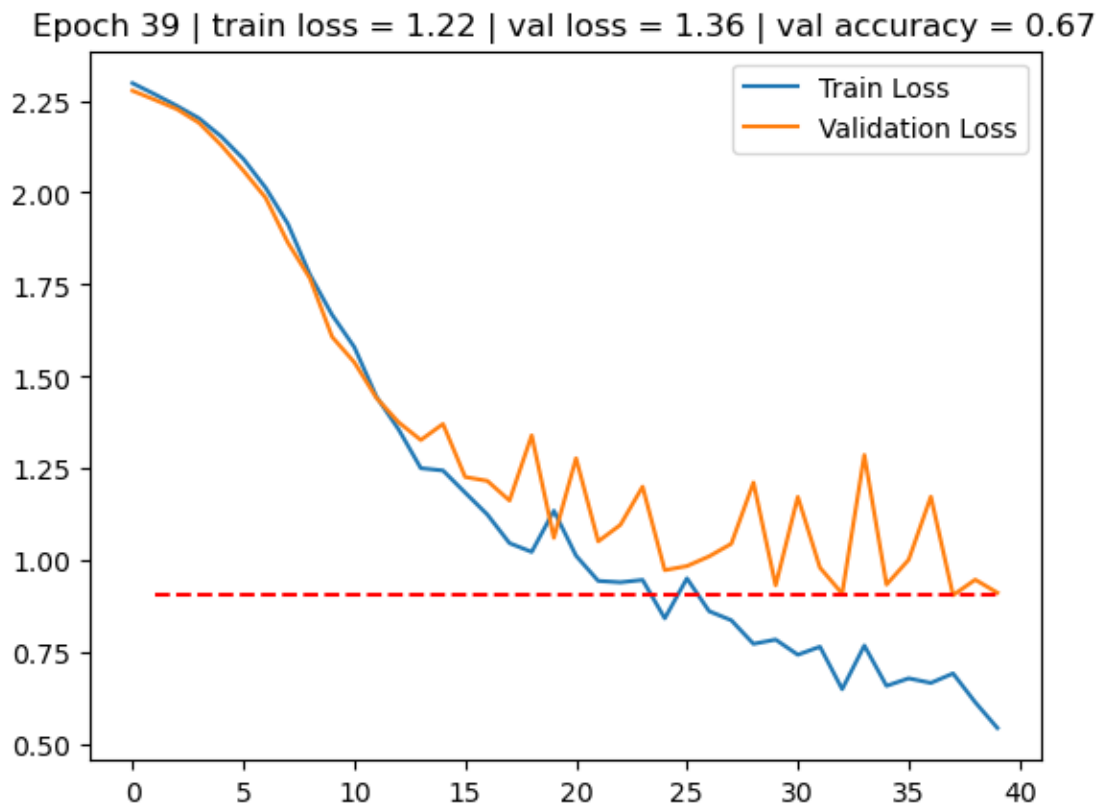
Next we will train a model. We will use the above defined hyperparameters, and a simple SGD optimizer. Furthermore we will print the training stats every epoch.

```

[20]: model = create_model(HIDDEN_DIMS)
optimizer = torch.optim.SGD(model.parameters(), lr=LR)

train_losses, val_losses = train(model,
                                train_loader,
                                val_loader,
                                optimizer,
                                n_epochs=40,
                                callbacks=[plot_train_and_val_loss_wrapper])

```



6 Early stopping

Early stopping is the most simple thing to prevent your final model to overfit: you simply track train and test errors and use a model checkpoint before your model started to overfit.

6.1 Task 3a) (10 P)

Implementing early stopping is pretty straight forward. Simply save your model each time you've reached a new best validation loss, and otherwise don't. To reduce clutter in your filesystem you can simply override the saved model each time. Saving and loading PyTorch models is described in [this guide](#). The function to load models is given.

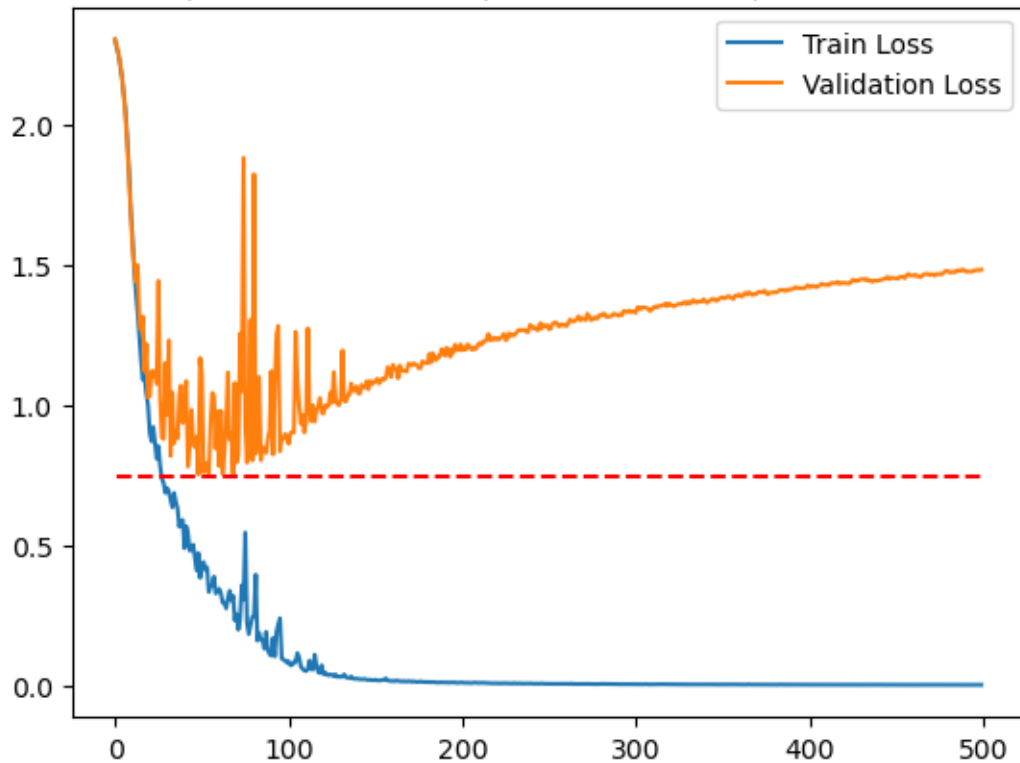
```
[23]: import os
def save_model_if_improved(model, epoch, train_losses, val_losses, val_acc,
    filename):
    # -----
    # TODO: Replace by your code
    # -----
    if max(val_losses) == val_losses[-1]:
        path = os.path.join(".", filename)
        torch.save(model.state_dict(), path)
    # -----

def load_model(model, model_file_path: str) -> nn.Module:
    model.load_state_dict(torch.load(model_file_path, weights_only=True))
    return model
```

This time we will train our model for more epochs, to show how it overfits. Your implemented function should stop saving the model once the model starts to overfit.

```
[25]: model = create_model(HIDDEN_DIMS)
optimizer = SGD(model.parameters(), lr=LR)
train_losses, val_losses = train(
    model,
    train_loader,
    val_loader,
    optimizer,
    n_epochs=EPOCHS,
    callbacks=[
        plot_train_and_val_loss_wrapper,
        partial(save_model_if_improved, filename="early_stopping.pth"),
    ],
)
```

Epoch 499 | train loss = 0.14 | val loss = 1.26 | val accuracy = 0.73



We now evaluate our model on the test set, to get a better estimate of the generalization error. As you can see we've named the model

```
[27]: pretrained_model = create_model(HIDDEN_DIMS)
load_model(pretrained_model, model_file_path="early_stopping.pth")
test_loss, test_accuracy = validate(pretrained_model, test_loader)

print(f"Early stopping model achieved test loss of {test_loss:.4f} and accuracy_
↪{test_accuracy:.4f}")
```

Early stopping model achieved test loss of 2.2915 and accuracy 0.2117

7 Dropout

Dropout is a technique where neurons are randomly set to zero, which introduces noise into the network, and helps to generalize better. Details are described in [this blogpost](#).

7.1 Task 4a) (20 P)

Implement a function similar to the one above where we build a standard feed forward network. But now, after each activation layer, add a `nn.Dropout()` layer with the dropout probability specified in the parameter `p`.

```
[30]: def create_model_with_dropout(hidden_dims: list[int], p: float):
    # -----
    # TODO: Replace by your code
    # -----
    model = SimpleMLPWithDropout(hidden_dims, p)
    # -----

    model.to(DEVICE)
    return model

class SimpleMLPWithDropout(SimpleMLP):

    def __init__(self, dims: list[int], p: float):
        super().__init__(dims)
        self.dropout = nn.Dropout(p)

    def forward(self, input: torch.Tensor):

        # -----
        # TODO: Replace by your code
        # -----
        out = self.flatten(input)
        for linear in self.linears[:-1]:
            out = self.dropout(self.activation(linear(out)))

        return self.linears[-1](out)
```

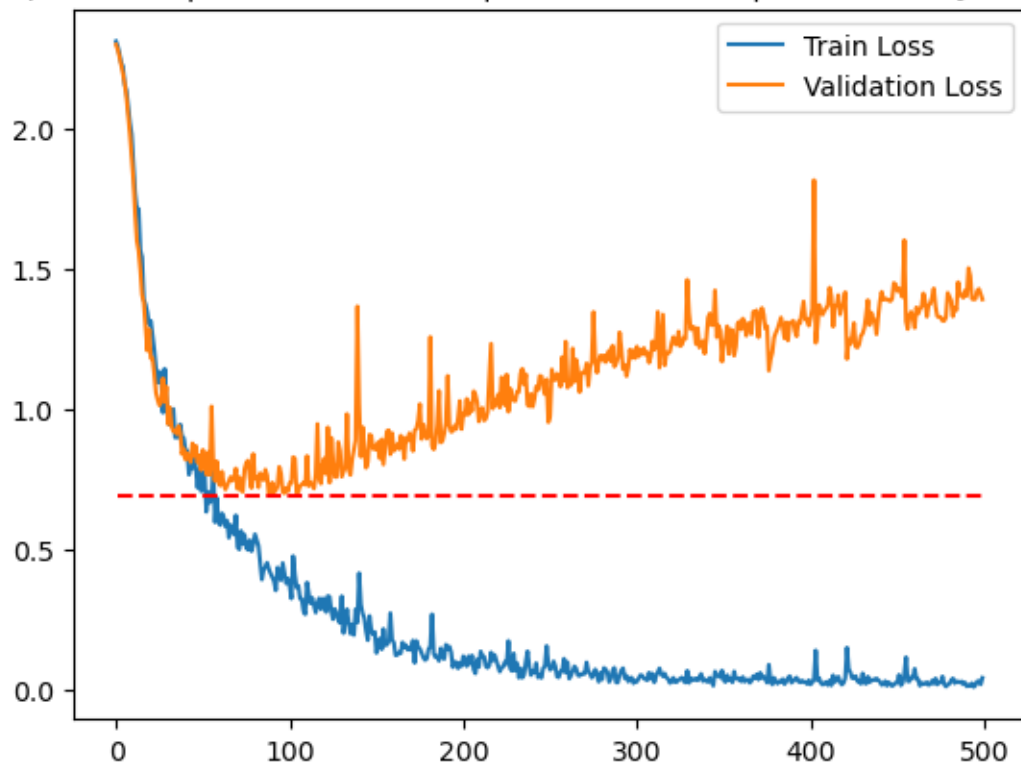
Next we will train a neural network with the exact same parameters as above, only that we now have added dropout layers.

```
[32]: DROPOUT = 0.25

model = create_model_with_dropout(HIDDEN_DIMS, p=DROPOUT)
optimizer = SGD(model.parameters(), lr=LR)

train_losses, val_losses = train(
    model,
    train_loader,
    val_loader,
    optimizer,
    n_epochs=EPOCHS,
    callbacks=[
        plot_train_and_val_loss_wrapper,
        partial(save_model_if_improved, filename="dropout.pth"),
    ],
)
```

Epoch 499 | train loss = 0.26 | val loss = 1.12 | val accuracy = 0.74



```
[33]: pretrained_model = create_model_with_dropout(HIDDEN_DIMS, DROPOUT)
      load_model(pretrained_model, "dropout.pth")
      test_loss, test_accuracy = validate(pretrained_model, test_loader)

      print(f"Dropout model achieved test loss of {test_loss:.4f} and accuracy of_
            ↳{test_accuracy:.4f}")
```

Dropout model achieved test loss of 2.2899 and accuracy of 0.2065

```
[ ]:
```